

Dashboard Financiero para Optimización de Portafolios

Prompt de Desarrollo y Especificaciones Técnicas

Objetivo Principal:

Desarrollar una aplicación profesional, escalable y modular para el análisis de acciones y construcción de portafolios de inversión.

1 Objetivo del Proyecto

Desarrollar una aplicación profesional utilizando **Python, Visual Studio Code y Streamlit**. La aplicación deberá permitir al usuario analizar acciones y construir un portafolio de inversión utilizando información histórica obtenida de Internet.

El desarrollo deberá seguir una arquitectura modular, evitando concentrar toda la lógica en un solo archivo. La prioridad es que el proyecto sea:

- Fácil de mantener.
- Fácil de ampliar.
- Profesional.
- Escalable.
- Bien documentado.

2 Tecnologías a Utilizar

El proyecto deberá desarrollarse utilizando el siguiente stack tecnológico:

Categoría	Herramientas / Librerías
Entorno y Core	Python 3.x, Visual Studio Code
Interfaz Gráfica	Streamlit
Manejo de Datos	Pandas, NumPy
Cálculos y Optimización	SciPy, CVXPY (para optimización)
Visualización	Plotly
Fuentes de Datos	yfinance
Exportación	OpenPyXL (exportación a Excel)

3 Estructura Inicial del Proyecto

Se proponen aproximadamente 15 archivos distribuidos en la siguiente estructura modular:

```
PortfolioDashboard/
├── app.py (Punto de entrada)
├── config.py (Configuraciones)
├── requirements.txt (Dependencias)
├── README.md (Documentación)
├── pages/
│   ├── Dashboard.py
│   ├── Portafolio.py
│   ├── Optimizacion.py
│   └── Configuracion.py
└── modules/
    ├── data.py
    ├── metrics.py
    ├── portfolio.py
    ├── optimization.py
    ├── charts.py
    ├── export.py
    └── helpers.py
```

3.1 Descripción de Archivos Raíz

app.py

Será el punto de entrada de toda la aplicación. Se encargará de inicializar Streamlit, configurar el menú lateral, cargar las páginas y configurar el tema general.

config.py

Contendrá toda la configuración del proyecto (periodo por defecto, benchmark, tasa libre de riesgo, colores, parámetros generales). Toda configuración deberá centralizarse aquí.

requirements.txt

Listado de todas las librerías necesarias para ejecutar el proyecto.

README.md

Documentación principal del proyecto explicando objetivo, instalación, ejecución, dependencias y estructura.

3.2 Carpeta pages/

Contendrá **únicamente la interfaz gráfica**, sin cálculos financieros.

- **Dashboard.py:** Pantalla principal con resumen, métricas, gráficos generales y rendimiento acumulado.
- **Portafolio.py:** Permite agregar/eliminar activos, definir pesos, seleccionar benchmark y elegir periodo.

- **Optimizacion.py:** Muestra frontera eficiente, Capital Allocation Line y portafolios óptimos (mínima varianza, máximo Sharpe).
- **Configuracion.py:** Configuraciones de usuario (fuente de datos, tasa, frecuencia, exportaciones).

3.3 Carpeta modules/

Aquí vivirá **toda la lógica del proyecto**. La interfaz únicamente llamará funciones de estos módulos.

- **data.py:** Obtiene información (inicialmente Yahoo Finance, extensible a Investing/Finviz) y limpia datos.
- **metrics.py:** Funciones independientes para rendimientos (diarios/mensuales/acumulado), Alpha de Jensen, Beta, Sharpe Ratio, Varianza, Desviación Estándar, VaR, correlaciones y covarianzas.
- **portfolio.py:** Construye el portafolio, calcula pesos, valor, rendimiento y riesgo del portafolio.
- **optimization.py:** Lógica de optimización (Markowitz, frontera eficiente, CAL, portafolios óptimos).
- **charts.py:** Genera gráficas con Plotly (evolución, rendimientos, pesos, frontera, CAL, heatmap, riesgo vs rendimiento).
- **export.py:** Exporta resultados a Excel/CSV (futuro: PDF, PowerPoint).
- **helpers.py:** Funciones auxiliares (fechas, validaciones, formatos, colores).

4 Flujo General de la Aplicación

Usuario → Selecciona empresas → Descarga datos históricos →
Limpieza → Cálculo rendimientos → Cálculo métricas →
Construcción portafolio → Optimización →
Generación gráficas → Visualización Streamlit → Exportación

5 Objetivos de la Primera Versión (MVP)

La primera versión deberá permitir:

1. Seleccionar uno o varios activos y descargar los últimos cinco años de información.
2. Calcular rendimientos (diarios/mensuales).
3. Calcular métricas clave: Alpha de Jensen, Beta, Sharpe Ratio, varianza, desviación estándar y VaR.
4. Construir automáticamente un portafolio.
5. Calcular la frontera eficiente y la Capital Allocation Line.
6. Mostrar resultados mediante gráficas interactivas.
7. Exportar resultados a Excel.

6 Buenas Prácticas de Desarrollo

Durante todo el proyecto deberán seguirse rigurosamente estas reglas:

- Mantener una arquitectura modular y evitar código duplicado.
- Crear funciones pequeñas y reutilizables.
- Separar **completamente** la lógica de negocio de la interfaz gráfica.
- Documentar todas las funciones.
- Utilizar nombres descriptivos para variables y funciones.
- Manejar adecuadamente los errores y validar entradas del usuario.
- Escribir código limpio y fácil de mantener.
- Desarrollar un módulo a la vez, validándolo antes de continuar.

Este documento sirve como base para la primera versión. Una vez estable, el MVP podrá evolucionar hacia una arquitectura más avanzada (simulación de Monte Carlo, modelos multifactoriales, backtesting, etc.).